

Los Sabrossos

Contents

1	STRINGS	3
1.1	Aho corasick	3
1.2	Hashing	6
1.3	Kmp	6
1.4	Suffix array	7
1.5	Trie	9
2	GRAPH ALGORITHMS	9
2.1	Dijkstra	9
2.2	Kruskal mst	10
2.3	Topological sort kahn algorithm	11
3	MATH	11
3.1	Binary exponentiation	11
3.2	Modular inverse	11
3.3	Pollard rho rabin miller	12
3.4	Sieve of eratosthenes	13
4	DATA STRUCTURES	13
4.1	Disjoint set union	13
4.2	Ordered set	14
4.3	Segment tree	15
4.4	Sparse table	15
5	FLOW	16
5.1	Dinic	16
6	UTILITIES	17
6.1	Merge sort	17
7	GEOMETRY	18
7.1	Convex hull	18
7.2	Point	18
8	TREE ALGORITHMS	19
8.1	Binary lifting	19
8.2	Euler tour	19
8.3	Lowest common ancestor	20
9	BIT MANIPULATION	20
9.1	Binary representation	20

1 STRINGS

1.1 Aho corasick

```
1 //AhoCorasick para saber si un string s existe en un texto T
2 const int E = 26;
3 const int N = 1e6+10;
4 int cur;
5 int nodoTerminal[N]; //se guarda el nodo terminal del string i
6 set<int>st; //se guarda que nodos son terminales que aparecen en el texto T
7
8 struct AC {
9     int nodes;
10    vector<int>suf, super;
11    vector<array<int, E>>go;
12    vector<bool>terminal;
13    AC(){
14        add_node();
15        nodes = 1;
16    }
17    void add_node(){
18        terminal.emplace_back();
19        go.emplace_back();
20        super.emplace_back();
21        suf.emplace_back();
22    }
23
24    void insert(string &s){
25        int pos = 0;
26        for(char ch : s){
27            int c = ch - 'a';
28            if(go[pos][c] == 0){
29                go[pos][c] = nodes++;
30                add_node();
31            }
32            pos = go[pos][c];
33        }
34        terminal[pos] = true;
35        nodoTerminal[cur] = pos;
36    }
37
38    void build(){
39        queue<int>Q;
40        Q.emplace(0);
41        while(!Q.empty()){
42            int u = Q.front();
43            Q.pop();
44            super[u] = (suf[u] == 0 or terminal[suf[u]] ? suf[u] : super[suf[u]]);
45            for(int c = 0; c < E; c++){
46                if(go[u][c]){
47                    int v = go[u][c];
48                    Q.emplace(v);
49                    suf[v] = u == 0 ? 0 : go[suf[u]][c];
50                }
51                else{
52                    go[u][c] = u == 0 ? 0 : go[suf[u]][c];
53                }
54            }
55        }
56    }
57 };
58
```

```

59 void mark(int u, AC &ac){
60     if(u == 0) return;
61     mark(ac.super[u], ac);
62     if(ac.terminal[u]){
63         st.insert(u);
64         ac.terminal[u] = false;
65     }
66     ac.super[u] = 0;
67 }
68
69 void solve(){
70     int n;
71     cin >> n;
72     AC ac;
73     repl(i,0,n){
74         string s;
75         cin >> s;
76         cur = i;
77         ac.insert(s);
78     }
79     string t; //texto grande
80     cin >> t;
81     ac.build();
82     int p = 0;
83     for(auto x : t){
84         int c = x - 'a';
85         p = ac.go[p][c];
86         mark(p, ac);
87     }
88     repl(i,0,n){
89         if(st.count(nodoTerminal[i])){
90             cout << "YES\n";
91         }
92         else{
93             cout << "NO\n";
94         }
95     }
96 }
97
98 //-----
99
100 //AhoCorasick para Frecuencias
101 const int E = 26;
102 const int N = 1e6+10;
103 int nodoTerminal[N];
104 int cur;
105 struct AC {
106     int nodes;
107     vector<int>suf, freq, by_level;
108     vector<bool>terminal;
109     vector<array<int,E>>go;
110
111     AC(){
112         add_node();
113         nodes = 1;
114     }
115     void add_node(){
116         terminal.emplace_back();
117         suf.emplace_back();
118         go.emplace_back();
119         freq.emplace_back();
120     }
121

```

```

122 void insert(string &s){
123     int pos = 0;
124     for(char ch:s){
125         int c = ch - 'a';
126         if(go[pos][c] == 0){
127             go[pos][c] = nodes++;
128             add_node();
129         }
130         pos = go[pos][c];
131     }
132     terminal[pos] = true;
133     nodoTerminal[cur] = pos;
134 }
135
136 void build(){
137     queue<int>Q;
138     Q.emplace(0);
139     while(!Q.empty()){
140         int u = Q.front();
141         Q.pop();
142         by_level.emplace_back(u);
143         for(int c = 0; c < E; c++){
144             if(go[u][c]){
145                 int v = go[u][c];
146                 Q.emplace(v);
147                 suf[v] = u == 0 ? 0 : go[suf[u]][c];
148             }
149             else{
150                 go[u][c] = u == 0 ? 0 : go[suf[u]][c];
151             }
152         }
153     }
154 }
155 };
156
157 void solve(){
158     int n;
159     cin >> n;
160     AC ac;
161     repl(i,0,n){
162         string s;
163         cin >> s;
164         cur = i;
165         ac.insert(s);
166     }
167     string t;
168     cin >> t;
169     ac.build();
170     int p = 0;
171     for(char ch:t){
172         int c = ch - 'a';
173         p = ac.go[p][c];
174         ac.freq[p]++;
175     }
176     for(int i = (int)ac.by_level.size()-1; i > 0; i--){
177         int x = ac.by_level[i];
178         ac.freq[ac.suf[x]] += ac.freq[x];
179     }
180     repl(i,0,n){
181         int termi = nodoTerminal[i];
182         cout << ac.freq[termi] << '\n';
183     }
184 }

```

1.2 Hashing

```
1 struct Hash {
2     long long P=1777771,MOD[2],PI[2];
3     vector<long long> h[2],pi[2];
4     Hash(string& s){
5         MOD[0]=999727999;MOD[1]=1070777777;
6         PI[0]=325255434;PI[1]=10018302;
7         for(int k = 0; k < 2; k++) h[k].resize(s.size()+1),pi[k].resize(s.size()+1);
8         for(int k = 0; k < 2; k++){
9             h[k][0]=0;pi[k][0]=1;
10            long long p=1;
11            for(int i = 1; i < s.size()+1; i++){
12                h[k][i]=(h[k][i-1]+p*s[i-1])%MOD[k];
13                pi[k][i]=(1LL*pi[k][i-1]*PI[k])%MOD[k];
14                p=(p*P)%MOD[k];
15            }
16        }
17    }
18    long long get(int s, int e){
19        long long h0=(h[0][e]-h[0][s]+MOD[0])%MOD[0];
20        h0=(1LL*h0*pi[0][s])%MOD[0];
21        long long h1=(h[1][e]-h[1][s]+MOD[1])%MOD[1];
22        h1=(1LL*h1*pi[1][s])%MOD[1];
23        return (h0<<32)|h1;
24    }
25 };
```

1.3 Kmp

```
1 vector<int> kmp(string s){
2     int n = s.size();
3     vector<int> pi(n);
4     for(int i = 1; i < n; i++){
5         int j = pi[i-1];
6         while(j > 0 and s[i] != s[j]){
7             j = pi[j-1];
8         }
9         if(s[i] == s[j]){
10            j++;
11        }
12        pi[i] = j;
13    }
14    return pi;
15 }
16 //Pattern Matching
17 vector<int> ocurrencia(string texto, string patron) {
18     vector<int> P = kmp(patron);
19     int k = 0;
20     vector<int> M;
21     repl(i,0,texto.size()) {
22         while (k > 0 and patron[k] != texto[i]) k = P[k - 1];
23         if (patron[k] == texto[i]) k++;
24         if (k == patron.size()) {
25             M.pb(i - k + 1);
26             k = P[k - 1];
27         }
28     }
29     return M;
30 }
31 }
```

```

32 //Automata
33 void genAut(string &s) {
34     int n = s.size();
35
36     vector<vector<int>> aut(n+1, vector<int>(26));
37     vector<int> pi = kmp(s);
38
39     for(int i = 0 ; i < n; i++){
40         aut[i][s[i]-'a'] = i+1;
41     }
42
43     for(int i = 0; i < n+1; i++){
44         for(int c = 0; c < 26; c++){
45             if (aut[i][c]) continue;
46             if (i < n and s[i] == 'a' + c) aut[i][c] = i+1;
47             else if (i > 0) aut[i][c] = aut[pi[i-1]][c];
48             else aut[i][c] = 0;
49         }
50     }
51 }

```

1.4 Suffix array

```

1 vector<int> suffix_array(string s) {
2     s += char(0);
3     const int n = s.size();
4     vector<int> a(n);
5     iota(a.begin(), a.end(), 0);
6     sort(a.begin(), a.end(), [&] (int i, int j){
7         return s[i] < s[j];
8     });
9     vector<int> mapping(n);
10    mapping[a[0]] = 0;
11    for(int i = 1; i < n; i++){
12        mapping[a[i]] = mapping[a[i-1]]+(s[a[i-1]] != s[a[i]]);
13    }
14    int len = 1;
15    vector<int> sbs(n);
16    vector<int> head(n);
17    vector<int> new_mapping(n);
18    while(len < n){
19        for(int i = 0; i < n; i++) sbs[i] = (a[i] + n - len) % n;
20        for(int i = n-1; i >= 0; i--) head[mapping[a[i]]] = i;
21        for(int i = 0; i < n; i++){
22            int x = sbs[i];
23            a[head[mapping[x]]++] = x;
24        }
25        new_mapping[a[0]] = 0;
26        for(int i = 1; i < n; i++){
27            if(mapping[a[i-1]] != mapping[a[i]]){
28                new_mapping[a[i]] = new_mapping[a[i-1]]+1;
29            }
30            else{
31                int pre = mapping[(a[i-1] + len)%n];
32                int cur = mapping[(a[i]+len) % n];
33                new_mapping[a[i]] = new_mapping[a[i-1]] + (pre!=cur);
34            }
35        }
36        len <= 1;
37        swap(mapping, new_mapping);
38    }
39    return vector<int>(a.begin()+1, a.end()); //ignorar a[0] que es el centinela

```



```

40 }
41
42 vector<int> lcp_array(vector<int>&a, string s){
43     const int n = s.size();
44     vector<int> rank(n);
45     for(int i = 0; i < n; i++) rank[a[i]] = i;
46     int k = 0;
47     vector<int> lcp(n);
48     for(int i = 0; i < n; i++){
49         if(rank[i] == n-1){
50             k = 0;
51             continue;
52         }
53         int j = a[rank[i]+1];
54         while(i+k < n and j+k < n and s[i+k] == s[j+k]) k++;
55         lcp[rank[i]] = k;
56         if(k) k--;
57     }
58     lcp.pop_back();
59     return lcp;
60 }
61
62 void solve(){
63     //Numero de distintos substr en un string n*(n+1)/2 - sumatoria del LCP
64     //verificar si un string es substring de otro con BS en SA
65     //encontrar el longest common substring en 2 strings(ESTA IMPLEMENTACION)
66     string s, t;
67     cin >> s >> t;
68     if(s.size() > t.size()){
69         swap(s, t);
70     }
71     int n = s.size();
72     int m = t.size();
73     string st = s+'#'+t;
74     vector<int> sa = suffix_array(st);
75     vector<int> lcp = lcp_array(sa, st);
76     // print(sa);
77     // print(lcp);
78     vector<int> tr(n+m);
79     repl(i,0,n+m){
80         if(sa[i] >= n and sa[i+1] >= n){
81             tr[i] = 0;
82             continue;
83         }
84         if(sa[i] < n and sa[i+1] < n){
85             tr[i] = 0;
86             continue;
87         }
88         int mn = min(sa[i], sa[i+1]);
89         int tam1 = mn + lcp[i];
90         // dbg(tam1);
91         tr[i] = lcp[i];
92         if(tam1 > n){
93             dbg(tam1);
94             tr[i] = lcp[i] - (tam1-n);
95         }
96     }
97     // print(tr);
98     int mx = *max_element(tr.begin(), tr.end());
99     repl(i,0,n+m){
100         // dbg(tr[i]);
101         if(tr[i] == mx){
102             string ans = st.substr(sa[i], mx);

```

```

103         cout << ans << '\n';
104         return;
105     }
106 }
107
108 }
109 //Sabrossus

```

1.5 Trie

```

1  const int E = 26;
2
3  struct AC {
4      int nodes;
5      vector<array<int, E>>go;
6      vector<bool>terminal;
7
8      AC(){
9          add_node();
10         nodes = 1;
11     }
12
13     void add_node(){
14         terminal.emplace_back();
15         go.emplace_back();
16     }
17
18     void insert(string &s){
19         int pos = 0;
20         for(char ch : s){
21             int c = ch - '0';
22             if(go[pos][c] == 0){
23                 go[pos][c] = nodes++;
24                 add_node();
25             }
26             pos = go[pos][c];
27         }
28         terminal[pos] = true;
29     }
30
31 };

```

2 GRAPH ALGORITHMS

2.1 Dijkstra

```

1  const int N = 1e5;
2
3  int vis[N];
4  vector<pair<int,int>> G[N];
5
6  void dijkstra(int x){
7      priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
8      pq.push({0,x});
9      vis[x] = 0;
10     while(!pq.empty()){
11         auto y = pq.top();
12         pq.pop();
13         int u = y.second;

```

```

14         int cost = y.first;
15         for(auto v : G[u]){
16             int cur = cost + v.second;
17             if(cur < vis[v.first]){
18                 vis[v.first] = cur;
19                 pq.push({cur,v.first});
20             }
21         }
22     }
23 }

```

2.2 Kruskal mst

```

1  const int N = 1e5;
2  struct DSU{
3      int num;
4      vector<int> parent;
5      vector<int> sizes;
6      DSU(){};
7      DSU(int n){
8          init(n);
9      }
10     void init(int n){
11         num = n;
12         parent.resize(n+1);
13         sizes.resize(n+1);
14         for(int i = 1; i <= n ; i++){
15             parent[i] = i;
16             sizes[i] = 1;
17         }
18     }
19     int find(int a){
20         if(a == parent[a]) return a;
21         return parent[a] = find(parent[a]);
22     }
23     void join(int a, int b){
24         int x = find(a);
25         int y = find(b);
26         if(x == y) return;
27         num--;
28         if(sizes[x] < sizes[y]){
29             parent[x] = y;
30             sizes[y] += sizes[x];
31         }
32         else{
33             parent[y] = x;
34             sizes[x] += sizes[y];
35         }
36     }
37 };
38 int n;
39 vector<pair<int,pair<int,int>>> List;
40 int Kruskal_MST(){
41     sort(List.begin(),List.end());
42     DSU dsu(n);
43     int mst = 0, t = 1;
44     for(int i = 0; i < List.size(); i++){
45         int w = List[i].first; //weight
46         int a = List[i].second.first; //at
47         int b = List[i].second.second; //to
48         if(dsu.find(a) != dsu.find(b)){
49             dsu.join(a,b);

```

```

50         mst+=w;
51         t++;
52     }
53     if(t == n) break;
54 }
55 return mst;
56 }

```

2.3 Topological sort kahn algorithm

```

1  const int N = 1e5;
2
3  int indegree[N];
4  vector<int> G[N];
5  int n;
6  vector<int> Kahn(){
7      queue<int> q;
8      vector<int> ans;
9      for(int i = 0; i < n; i++){
10         if(indegree[i] == 0) q.push(i);
11     }
12     while (!q.empty()){
13         int v = q.front();
14         q.pop();
15         ans.push_back(v);
16         for(int u : G[v]){
17             indegree[u]--;
18             if(indegree[u] == 0){
19                 q.push(u);
20             }
21         }
22     }
23     //if(ans.size() != n) grafo no DAG
24     return ans;
25 }

```

3 MATH

3.1 Binary exponentiation

```

1  long long pote(long long a, long long b, long long mod){
2      long long ans = 1ll;
3      while(b){
4          if(b&1) ans = (ans * a)%mod;
5          a = (a * a)%mod;
6          b>>=1;
7      }
8      return ans;
9  }
10 // a^-1 = a^mod-2 = pote(a,mod-2);

```

3.2 Modular inverse

```

1  const long long N = 1e6 + 10;
2  const int MOD = 1e9+7;
3  long long fact[N];
4  long long invfact[N];

```

```

5 void init(){
6     fact[0] = 1;
7     for(long long i = 1; i < N; i++)
8         fact[i] = fact[i-1] * i % MOD;
9
10    invfact[N-1] = pote(fact[N-1], MOD-2);
11
12    for(long long i = N-2; i >= 0; i--)
13        invfact[i] = invfact[i+1] * (i+1) % MOD;
14 }
15
16 long long nCk(long long n, long long k){
17     if(k < 0 || k > n) return 0;
18     return fact[n] * invfact[k] % MOD * invfact[n-k] % MOD;
19 }

```

3.3 Pollard rho rabin miller

```

1 map<long long, long long> fact;
2
3 long long mcd(long long a, long long b) {
4     a = abs(a);
5     b = abs(b);
6     while (a != 0) {
7         long long r = b % a;
8         b = a;
9         a = r;
10    }
11    return b;
12 }
13 //Rabin Miller
14 bool primo(long long n) {
15     if (n < 2) return false;
16     if (n == 2) return true;
17     long long D = n - 1, S = 0;
18     while (D % 2 == 0) {
19         D /= 2;
20         S++;
21     }
22     static const long long STEPS = 1611;
23     for(long long pasos = 0; pasos < STEPS; pasos++){
24         const long long A = 1 + rand() % (n - 1);
25         long long M = pote(A, D, n);
26         if (M == 1 || M == (n - 1)) goto next;
27         for(long long k = 0; k < S-1; k++){
28             M = ((M) * M) % n;
29             if (M == (n - 1)) goto next;
30         }
31         return false;
32     next:;
33     }
34     return true;
35 }
36
37 //Rho de pollard
38 long long factor(long long n) {
39     static long long A, B;
40     A = 1 + rand() % (n - 1);
41     B = 1 + rand() % (n - 1);
42     #define fun(x) ((x) * (x + B)) % n + A
43
44     long long x = 2, y = 2, d = 1;

```

```

45     while (d == 1 || d == -1) {
46         x = fun(x);
47         y = fun(fun(y));
48         d = mcd(x - y, n);
49     }
50     return abs(d);
51 }
52
53 void factorize(long long n){
54     assert(n > 0);
55     while (n > 1 && !primo(n)) {
56         long long fx;
57         do {
58             fx = factor(n);
59         } while (fx == n);
60
61         n /= fx;
62         factorize(fx);
63
64         for (auto &it : fact) {
65             while (n % it.first == 0) {
66                 n /= it.first;
67                 it.second++;
68             }
69         }
70     }
71     if (n > 1) fact[n]++;
72 }

```

3.4 Sieve of eratosthenes

```

1  const int N = 1e7;
2  int sieve[N];
3  void make(){
4      for(int i = 0; i < N; i++) sieve[i] = 1;
5      sieve[0] = sieve[1] = 0;
6      for (int i = 2; i < N; i++)
7      {
8          if(sieve[i]){
9              for (int j = i*i; j < N; j+=i)
10                 {
11                     sieve[j] = 0;
12                 }
13             }
14         }
15     }

```

4 DATA STRUCTURES

4.1 Disjoint set union

```

1  const int N = 1e5;
2  struct DSU{
3      int num;
4      vector<int> parent;
5      vector<int> sizes;
6      DSU(){};
7      DSU(int n){
8          init(n);

```

```

9      }
10     void init(int n){
11         num = n;
12         parent.resize(n+1);
13         sizes.resize(n+1);
14         for(int i = 1; i <= n ; i++){
15             parent[i] = i;
16             sizes[i] = 1;
17         }
18     }
19     int find(int a){
20         if(a == parent[a]) return a;
21         return parent[a] = find(parent[a]);
22     }
23     void join(int a, int b){
24         int x = find(a);
25         int y = find(b);
26         if(x == y) return;
27         num--;
28         if(sizes[x] < sizes[y]){
29             parent[x] = y;
30             sizes[y] += sizes[x];
31         }
32         else{
33             parent[y] = x;
34             sizes[x] += sizes[y];
35         }
36     }
37 }
38 };

```

4.2 Ordered set

```

1  #include <bits/stdc++.h>
2  #include <ext/pb_ds/assoc_container.hpp>
3  #include <ext/pb_ds/tree_policy.hpp>
4
5  using namespace std;
6  using namespace __gnu_pbds;
7
8  typedef tree<int, null_type, less<int>, rb_tree_tag, tree_order_statistics_node_update>
9      ordered_set;
10
11 void solve() {
12     ordered_set st;
13
14     st.insert(10);
15     st.insert(30);
16     st.insert(20);
17
18     // 1. Elemento en la posición K (0-indexed, de menor a mayor)
19     // find_by_order devuelve un iterador (puntero). Usamos '*' para el valor.
20     auto it = st.find_by_order(1);
21     if (it != st.end()) {
22         int elemento = *it; // Devuelve 20 de forma segura
23     }
24
25     // 2. Cuantos elementos son estrictamente menores que X
26     int menores_que_25 = st.order_of_key(25); // Devuelve 2 (el 10 y el 20)
27 }
28 // Sabrossus

```

4.3 Segment tree

```
1  const int N = 1e5;
2  struct info{
3      int num;
4  };
5  info ST[4*N];
6  info ope(info u, info v){
7      return {u.num+v.num};
8  }
9  void build(int in, int L, int R){
10     if(L == R){
11         cin >> ST[in].num;
12     }
13     else{
14         int mid = (L+R)>>1;
15         build((2*in),L,mid);
16         build((2*in)+1,mid+1,R);
17         ST[in] = ope(ST[(2*in)], ST[(2*in)+1]);
18     }
19 }
20 void update(int in, int L, int R, int pos, int val){
21     if(L == R){
22         ST[in].num = val;
23     }
24     else{
25         int mid = (L+R)>>1;
26         if(pos <= mid){
27             update(2*in,L,mid,pos,val);
28         }
29         else{
30             update((2*in)+1,mid+1,R,pos,val);
31         }
32         ST[in] = ope(ST[2*in], ST[(2*in)+1]);
33     }
34 }
35 info get(int in, int L, int R, int l, int r){
36     if(r < L or R < l) return {0};
37     if(l <= L and R <= r) return ST[in];
38     int mid = (L+R)>>1;
39     info left = get(2*in,L,mid,l,r);
40     info right = get((2*in)+1,mid+1,R,l,r);
41     return ope(left,right);
42 }
43 // v = {1 2 3 4 5}
44 // get(1, 0, n-1, 1, 3) = 2 + 3 + 4 = 9
```

4.4 Sparse table

```
1  const int N = 1e5;
2  const int LOG = 20;
3  int v[N]; //values
4  int st[N][LOG]; //Sparse Table
5  int lg[N]; // log values
6  int n;
7  void build(){
8      for(int i = 0; i < n; i++) st[i][0] = v[i];
9
10     for(int i = 1; i < LOG; i++){
11         for(int j = 0; j < n - (1<<i) + 1; j++){
12             st[j][i] = min(st[j][i-1],st[j+(1<<(i-1))][i-1]);
```



```

13     }
14 }
15 lg[1] = 0;
16 for(int i = 2; i <= n; i++) lg[i] = lg[i/2] + 1;
17 }
18 int query(int L, int R){
19     int k = lg[R-L+1];
20     return min(st[L][k], st[R-(1<<k)+1][k]);
21 }
22 // v = [5, 2, 4, 7, 1, 3]
23 // query(1, 4) → [2, 4, 7, 1]

```

5 FLOW

5.1 Dinic

```

1 struct Dinic
2 {
3     const int INF = 1e9;
4
5     struct flowEdge
6     {
7         int to, rev, f, cap;
8     };
9
10    vector<vector<flowEdge>> G;
11    vector<int> work, lvl, Q;
12
13    int S, T, nodes;
14
15    Dinic(){
16    Dinic(int n, int s, int t){
17        S = s;
18        T = t;
19        nodes = n;
20        G.clear();
21        G.resize(n);
22        Q.resize(n);
23    }
24    void addEdge(int st, int en, int cap){
25        flowEdge A = {en, (int)G[en].size(), 0, cap};
26        flowEdge B = {st, (int)G[st].size(), 0, 0};
27        G[st].pb(A);
28        G[en].pb(B);
29    }
30    int dfs(int v, int f){
31        if(v == T or f == 0) return f;
32        for(int &i = work[v]; i < G[v].size(); i++){
33            flowEdge &e = G[v][i];
34            int u = e.to;
35            if(e.cap <= e.f or lvl[u] != lvl[v] + 1) continue;
36            int df = dfs(u, min(f, e.cap - e.f));
37            if(df){
38                e.f += df;
39                G[u][e.rev].f -= df;
40                return df;
41            }
42        }
43        return 0;
44    }
45    bool bfs(){

```

```

46     int qt = 0;
47     Q[qt++] = S;
48     lvl.assign(nodes,-1);
49     lvl[S] = 0;
50     for(int i = 0; i < qt; i++){
51         int u = Q[i];
52         for(flowEdge &e : G[u]){
53             int v = e.to;
54             if(e.cap <= e.f or lvl[v] != -1) continue;
55             lvl[v] = lvl[u] + 1;
56             Q[qt++] = v;
57         }
58     }
59     return lvl[T] != -1;
60 }
61 int maxFlow(){
62     int flow = 0;
63     while(bfs()){
64         work.assign(nodes,0);
65         while(true){
66             int df = dfs(S,INF);
67             if(df == 0) break;
68             flow += df;
69         }
70     }
71     return flow;
72 }
73
74 };

```

6 UTILITIES

6.1 Merge sort

```

1 // Rango de uso: 0-indexed.
2 // Llamar en el solve como: mergeSort(v, 0, n - 1);
3 ll ans = 0;
4
5 void mergeSort(vector<ll> &v, int low, int high) {
6     if (low == high) return;
7
8     int mid = (low + high) >> 1;
9     mergeSort(v, low, mid);
10    mergeSort(v, mid + 1, high);
11
12    queue<ll> L, H;
13    for (int i = low; i < mid + 1; i++) {
14        L.push(v[i]);
15    }
16    for (int i = mid + 1; i < high + 1; i++) {
17        H.push(v[i]);
18    }
19
20    for (int i = low; i < high + 1; i++) {
21        if (L.size() == 0) {
22            v[i] = H.front();
23            H.pop();
24        }
25        else if (H.size() == 0) {
26            v[i] = L.front();
27            L.pop();

```

```

28     }
29     else {
30         if (L.front() <= H.front()) {
31             v[i] = L.front();
32             L.pop();
33         }
34         else {
35             v[i] = H.front();
36             H.pop();
37             ans += L.size(); // Inversion detectada
38         }
39     }
40 }
41 }

```

7 GEOMETRY

7.1 Convex hull

```

1 // Devuelve sentido horario
2 vector<point> hull(vector<point> p)
3 {
4     int n = p.size();
5     vector<point> h;
6     sort(p.begin(), p.end());
7     for(int i = 0; i < n; i++){
8         while(h.size() >= 2 and p[i].left(h[h.size() - 2], h.back()) < 0){
9             h.pop_back();
10        }
11        h.push_back(p[i]);
12    }
13    h.pop_back();
14    int k = h.size();
15    for(int i = n-1; i >= 0; i--){
16        {
17            while(h.size() >= k + 2 and p[i].left(h[h.size() - 2], h.back()) < 0){
18                h.pop_back();
19            }
20            h.pb(p[i]);
21        }
22        h.pop_back();
23    }
24    return h;
25 }

```

7.2 Point

```

1 struct point
2 {
3     int x, y;
4     point() {}
5     point(int x, int y): x(x), y(y) {}
6     point operator -(point p) {return point(x - p.x, y - p.y);}
7     point operator +(point p) {return point(x + p.x, y + p.y);}
8     int sq() const{return x * x + y * y;}
9     double abs() const {return sqrt(sq());}
10    int operator ^(point p) {return x * p.y - y * p.x;}
11    int operator *(point p) {return x * p.x + y * p.y;}
12    point operator *(int a) {return point(x * a, y * a);}
13    bool operator <(const point& p) const {return x == p.x ? y < p.y : x < p.x;}

```

```

14     int left(point a, point b) {
15         return ((b - a) ^ (*this - a)); // 0, -, +
16     }
17 };
18 ostream& operator<<(ostream& os, const point& p) {
19     return os << "(" << p.x << "," << p.y << ")\n";
20 }
21
22
23 void polarSort(vector<point>& v) {
24     sort(v.begin(), v.end(), [] (point a, point b) {
25         const point origin{0, 0};
26         bool ba = a < origin, bb = b < origin;
27         if (ba != bb) { return ba < bb; }
28         return (a^b) > 0;
29     });
30 }

```

8 TREE ALGORITHMS

8.1 Binary lifting

```

1  const int N = 1e5;
2  const int LOG = 20;
3  int up[N][LOG];
4  vector<int> G[N];
5  //Find parents
6  void dfs(int u){
7      for(int v : G[u]){
8          up[v][0] = u;
9          for(int j = 1; j < LOG; j++){
10             up[v][j] = up[up[v][j-1]][j-1];
11         }
12         dfs(v);
13     }
14 }
15 int get_K_parent(int a, int k){
16     for(int j = LOG-1; j >= 0; j++){
17         if(k && (1<<j)){
18             a = up[a][j];
19         }
20     }
21     return a;
22 }

```

8.2 Euler tour

```

1  const int N = 1e5 + 10;
2  vector<int> path;
3  vector<int> G[N];
4  void EulerTour(int node, int parent){
5      path.pb(node);
6      for(int u : G[node]){
7          if(u != parent){
8              EulerTour(u, node);
9          }
10     }
11     path.pb(node);
12 }

```

```

13
14 const int N = 1e5 + 10;
15 int tin[N];
16 int tout[N];
17 int flat[N];
18 int timer = 1;
19 void EulerTour(int node, int parent){
20     tin[node] = timer;
21     flat[timer] = node;
22     timer++;
23     for(int u : G[node]){
24         if(u != parent){
25             EulerTour(u, node);
26         }
27     }
28     tout[node] = timer;
29 }

```

8.3 Lowest common ancestor

```

1  const int N = 1e5;
2  const int LOG = 20;
3  vector<int> G[N];
4  int depth[N];
5  int up[N][LOG];
6  // Assign levels and get parents
7  void dfs(int u){
8      for(int v : G[u]){
9          depth[v] = depth[u] + 1;
10         up[v][0] = u;
11         for(int j = 1; j < LOG; j++){
12             up[v][j] = up[up[v][j-1]][j-1];
13         }
14         dfs(v);
15     }
16 }
17 int get_lca(int a, int b){
18     if(depth[a] < depth[b]) swap(a,b);
19     int k = depth[a] - depth[b];
20     for(int j = LOG-1; j >= 0; j--){
21         if(k & (1<<j)){
22             a = up[a][j];
23         }
24     }
25     if(a == b) return a;
26     for(int j = LOG-1; j >= 0; j--){
27         if(up[a][j] != up[b][j]){
28             a = up[a][j];
29             b = up[b][j];
30         }
31     }
32     return up[a][0];
33 }

```

9 BIT MANIPULATION

9.1 Binary representation

```
1 string tobit(long long x) {  
2     string a(64, '0');  
3     for (int i = 63; i >= 0; --i) {  
4         if (x & 1) a[i] = '1';  
5         x >>= 1;  
6     }  
7     return a;  
8 }
```